

```
// ----- ScoreSegmentCounter.h -----
// ScoreSegmentCounter.h
#ifndef SCORE_SEGMENT_COUNTER_H
#define SCORE_SEGMENT_COUNTER_H

#include <array>

// ScoreSegmentCounter counts how many scores fall into each 10-point segment.
// Segment k means:  $10 * k + 1 \leq \text{score} \leq 10 * k + 10$ , where k is 0..9.
class ScoreSegmentCounter
{
public:
    // Initialize the object with an integer score array and its element count.
    ScoreSegmentCounter(const int scores[], int n);

    // Return the number of scores in segment k.
    // Valid k values are 0..9.
    int operator[](int k) const;

private:
    std::array<int, 10> counts; // counts[k] stores the number of scores in segment k.
};

#endif

// ----- ScoreSegmentCounter.cpp -----
// ScoreSegmentCounter.cpp
#include "ScoreSegmentCounter.h"

#include <stdexcept>

ScoreSegmentCounter::ScoreSegmentCounter(const int scores[], int n)
{
    counts.fill(0);

    for (int i = 0; i < n; ++i)
    {
        int score = scores[i];

        // This problem counts the segments 1-10, 11-20, ..., 91-100.
        // Scores outside 1..100 are ignored.
        if (score >= 1 && score <= 100)
        {
            int segment = (score - 1) / 10;
            ++counts[segment];
        }
    }
}

int ScoreSegmentCounter::operator[](int k) const
{
    if (k < 0 || k >= 10)
    {
        throw std::out_of_range("segment index must be from 0 to 9");
    }

    return counts[k];
}
```

```
// ----- main.cpp -----
// main.cpp
#include "ScoreSegmentCounter.h"

#include <iostream>

int main()
{
    int scores[] = {100, 97, 93, 88, 76, 75, 61, 60, 59, 45, 31, 20, 10, 1, 0};
    int n = sizeof(scores) / sizeof(scores[0]);

    ScoreSegmentCounter obj(scores, n);

    for (int k = 0; k < 10; ++k)
    {
        int left = 10 * k + 1;
        int right = 10 * k + 10;
        std::cout << left << "-" << right << ": " << obj[k] << std::endl;
    }

    return 0;
}
```